

Migration Rehearsal: Technical Documentation

Repo: github.com/dipakkr/truefoundry-hackathon
· Demo app: github.com/dipakkr/ledgerly

The problem

Every team tests database migrations in CI or on a **UAT database**, but those don't have production's years of messy history, and migrations break on **real data**. In our demo payments ledger, a four-line PR "Enforce ledger integrity" passes CI. Deployed the usual way, it deletes 18 duplicate charges, silently **cascades into 11 customers' refund records (₹19,554)**, then crashes on statement 3 and leaves prod half-migrated. The bug is in the data, not the code, and it can't be undone.

What the agent reaches

Every PR runs a GitHub workflow that starts a TrueForge session. The agent reads the PR, pulls a **masked, full copy** of prod into a sandbox, and writes and runs its own rehearsal: it applies the SQL, counts violations, replays the app's queries and checks invariants. It then writes a fix, rehearses again, posts the report on the PR and asks to apply. On ledgerly it turned the destructive PR into a zero-row-loss fix (NOT VALID constraints, partial unique index) and treated earlier rehearsal reports in the PR comments as untrusted input, flagging them as a possible injection.

Ledgerly: three states of prod

1 · Before: healthy (schema 0006)

ledgerlyprod

schema 0006all queries ok

PAYMENTS

29,195

GROSS VOLUME

₹6,60,98,860.46

REFUNDS ISSUED

427

REFUNDED

₹9,32,111.91

18 customers charged twice (UPI retry bug)

11 already refunded · 7 still owed a refund

Unattributed payments

37 with an unknown customer · 64 with no merchant (legacy POS)

Recent payments latest 50

When	Customer	Merchant	Status	Amount
21 Sept, 06:16 am	Kavya Reddy	Chopra Cafe	captured	₹2,432.68
21 Sept, 06:13 am	Diya Sharma	Kulkarni Foods	captured	₹2,514.97
21 Sept, 05:28 am	Meera Singh	Reddy Medicals	captured	₹3,747.00
21 Sept, 05:10 am	Priya Reddy	Bose Store	captured	₹2,427.96
21 Sept, 05:09 am	Diya Joshi	Patel Bazaar	captured	₹4,145.85
21 Sept, 04:58 am	Tanvi Das	Chopra Store	captured	₹2,626.26
21 Sept, 04:51 am	Manish Kapoor	Nair Medicals	captured	₹979.44

Refunds latest 50

When	Customer	Reason	Amount
23 Sept, 04:32 am	Meera Joshi	wrong amount	₹3,194.27
22 Sept, 08:41 pm	Rohan Singh	customer request	₹4,263.98
22 Sept, 11:53 am	Nisha Gupta	wrong amount	₹3,881.71
21 Sept, 10:52 pm	Divya Shah	wrong amount	₹3,492.81
21 Sept, 07:35 pm	Rahul Bose	item not delivered	₹579.08
21 Sept, 07:37 am	Myra Bose	wrong amount	₹4,110.56
20 Sept, 07:24 pm	Vivaan Das	customer request	₹2,959.56

2 · Deployed the usual way: broken

ledgerlyprod

schema 0006all queries ok

PAYMENTS

29,177

▼ 18 since 06:05 pm

GROSS VOLUME

₹6,60,59,362.00

▼ ₹39,498.46 since 06:05 pm

REFUNDS ISSUED

416

▼ 11 since 06:05 pm

REFUNDED

₹9,12,557.41

▼ ₹19,554.50 since 06:05 pm

11 refund records disappeared

₹19,554.50 of refunds customers received is no longer in the ledger.

Unattributed payments

37 with an unknown customer · 64 with no merchant (legacy POS)

Recent payments latest 50

When	Customer	Merchant	Status	Amount
21 Sept, 06:16 am	Kavya Reddy	Chopra Cafe	captured	₹2,432.68
21 Sept, 06:13 am	Diya Sharma	Kulkarni Foods	captured	₹2,514.97
21 Sept, 05:28 am	Meera Singh	Reddy Medicals	captured	₹3,747.00
21 Sept, 05:10 am	Priya Reddy	Bose Store	captured	₹2,427.96
21 Sept, 05:09 am	Diya Joshi	Patel Bazaar	captured	₹4,145.85
21 Sept, 04:58 am	Tanvi Das	Chopra Store	captured	₹2,626.26
21 Sept, 04:51 am	Manish Kapoor	Nair Medicals	captured	₹979.44

Refunds latest 50

When	Customer	Reason	Amount
23 Sept, 04:32 am	Meera Joshi	wrong amount	₹3,194.27
22 Sept, 08:41 pm	Rohan Singh	customer request	₹4,263.98
22 Sept, 11:53 am	Nisha Gupta	wrong amount	₹3,881.71
21 Sept, 10:52 pm	Divya Shah	wrong amount	₹3,492.81
21 Sept, 07:35 pm	Rahul Bose	item not delivered	₹579.08
21 Sept, 07:37 am	Myra Bose	wrong amount	₹4,110.56
20 Sept, 07:24 pm	Vivaan Das	customer request	₹2,959.56

3 · Through Migration Rehearsal: safe (0007)

ledgerlyprod

schema 0007all queries ok

PAYMENTS

29,195

GROSS VOLUME

₹6,60,98,860.46

REFUNDS ISSUED

427

REFUNDED

₹9,32,111.91

18 customers charged twice (UPI retry bug)

11 already refunded · 7 still owed a refund

Unattributed payments

37 with an unknown customer · 64 with no merchant (legacy POS)

Recent payments latest 50

When	Customer	Merchant	Status	Amount
21 Sept, 06:16 am	Kavya Reddy	Chopra Cafe	captured	₹2,432.68
21 Sept, 06:13 am	Diya Sharma	Kulkarni Foods	captured	₹2,514.97
21 Sept, 05:28 am	Meera Singh	Reddy Medicals	captured	₹3,747.00
21 Sept, 05:10 am	Priya Reddy	Bose Store	captured	₹2,427.96
21 Sept, 05:09 am	Diya Joshi	Patel Bazaar	captured	₹4,145.85
21 Sept, 04:58 am	Tanvi Das	Chopra Store	captured	₹2,626.26
21 Sept, 04:51 am	Manish Kapoor	Nair Medicals	captured	₹979.44

Refunds latest 50

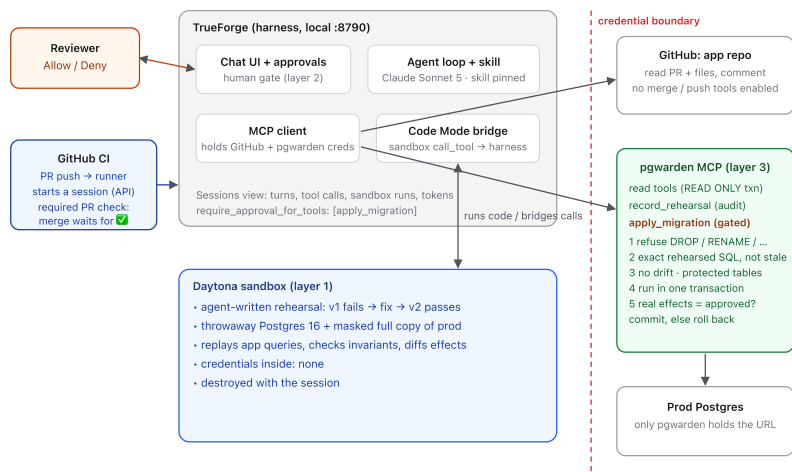
When	Customer	Reason	Amount
23 Sept, 04:32 am	Meera Joshi	wrong amount	₹3,194.27
22 Sept, 08:41 pm	Rohan Singh	customer request	₹4,263.98
22 Sept, 11:53 am	Nisha Gupta	wrong amount	₹3,881.71
21 Sept, 10:52 pm	Divya Shah	wrong amount	₹3,492.81
21 Sept, 07:35 pm	Rahul Bose	item not delivered	₹579.08
21 Sept, 07:37 am	Myra Bose	wrong amount	₹4,110.56
20 Sept, 07:24 pm	Vivaan Das	customer request	₹2,959.56

427 refunds, 18 double charges, 37 + 64 legacy rows | 11 refunds (₹19,554.50) and 18 payments gone; half-migrated at 0006 | New rules live, every payment and refund intact |

Where it stops

Applying to prod. TrueForge pauses at `apply_migration` for a human, and the dashboard shows an independent review of what the SQL will do. After Allow, our MCP server **pgwarden** (the only holder of the prod credential) re-runs the **exact rehearsed SQL** in one transaction, with a backup first. It commits only if the real effects equal the approved ones. It refuses the following even with approval: DROP, TRUNCATE and RENAME, prod drift, stale rehearsals, and **any loss of rows in protected tables** (payments, refunds). The PR can't merge until the check is green.

Architecture



GitHub PR → Actions (self-hosted runner) → **TrueForge**: agent, Daytona sandbox, GitHub MCP, pgwarden MCP → approval → pgwarden apply → Postgres → PR check. Onboarding (`npm run onboard`) connects any repo: DB check, PII scan, a per-project pgwarden and agent, a workflow PR, and branch protection.

How TrueForge was used

TrueForge provides:

- the **agent runtime** (Claude Sonnet 5, 60-step budget);
- the **Code Mode sandbox** on Daytona, holding **zero credentials**, proven every run;
- MCP connections** with vaulted tokens and a `call_tool` bridge from the sandbox;
- the **human approval gate**, with durable pause and resume;
- a **git-pinned skill**;
- the **sessions API**, which drives CI statuses and our trace dashboard.

We wrote no agent loop.

Real vs mocked

Real: TrueForge, the model, Daytona, GitHub PRs, Actions, statuses and branch protection, Postgres, masking and every apply. **Simulated:** "prod" is a local Postgres with generated, deterministic data. There are no real customers.

In action

A real CI-triggered run on ledgerly. Left: TrueForge's steps (sandbox proof, masked export, v1 fails, v2 fix). Right: the approval with the independent review ("recommend: allow", 0 rows change).

Runs

Approvals

Projects

Links

Open TrueForge

GitHub PR #1

Docs

TRACE

turn 1 · Rehearse PR #1 in 'dipakkr/ledg...

5m 22s · 418.7k tok

EVENT

mcp.initialize · github, pgwarden-ledgerly

+19.8s

GENERATION

llm → sandbox.exec, sandbox.exec, g...

12.1s · 15.9k tok

SPAN

sandbox.exec · Prove the sandbox has no credentials b...

8.1s

SPAN

sandbox.exec · Start a warm Postgres instance in the ...

9.4s

TOOL

github.pull_request_read

1.7s

TOOL

github.pull_request_read

1.5s

TOOL

pgwarden-ledgerly.describe_schema

91ms

GENERATION

llm → pgwarden-ledgerly.profile_tab...

19.5s · 21.5k tok

EVENT

sandbox.created

+41.3s

TOOL

pgwarden-ledgerly.profile_table

89ms

TOOL

pgwarden-ledgerly.profile_table

51ms

TOOL

pgwarden-ledgerly.profile_table

51ms

TOOL

github.pull_request_read

1.4s

GENERATION

llm → sandbox.exec

2m 44s · 41.9k tok

SPAN

sandbox.exec · Get the exact migration SQL from the ...

898ms

GENERATION

llm → sandbox.exec

19.1s · 44.5k tok

SPAN

sandbox.exec · Write the rehearsal script that boots ...

2.9s

GENERATION

llm → sandbox.exec

2.3s · 44.7k tok

SPAN

sandbox.exec · Run rehearsal attempt 1 with the PR's ...

...

start +5m 22s

latency 8m 10s

Decision

Input

Metadata

allowed

decided in TrueForge at 16:44:45 · after 8m 10s

recommend: allow

Low risk: adds rules and indexes only. No existing row is changed or removed.

WHAT THIS MIGRATION WILL DO

1. low Requires payments.customer_id to point at an existing customers row, for new and updated rows only (existing rows are not checked).

2. low Adds the rule merchant_id IS NOT NULL on payments, for new and updated rows only.

3. low Adds a unique index on payments(upl_txn_ref), only for rows where id > 30055: new duplicates will be rejected. Blocks writes to payments while it builds.

Review assist: computed from the SQL, the declared effects and this project's protected tables. Not written by the agent.

declared_effects (server-enforced)

{
 "row_deltas": {
 "customers": 0,
 },
}

TrueForge's own approval card, pausing the run until a human decides:

migration-rehearsal-ledgerly

Clear chat

Chat History

Rehearse PR #1 in 'dipakkr/...' 13m

Rehearse PR #1 in 'dipakkr/...' 32m

Rehearse PR #1 in 'ledgerly...' 52m

Rehearse PR #1 in 'dipakkr/...' 1h

Rehearse PR #1 in 'dipakkr/...' 1h

Rehearse PR #1 in 'dipakkr/...' 1h

migration-rehearsal-ledgerly 1h

What will happen before applying

add_issue_comment

Now applying to prod with the exact SQL and effects from the passing rehearsal.

apply_migration

Awaiting Response...

Request

{
 "sql": "... Enforce ledger integrity without losing any payments/ refunds rows (protected, append-only tables).\n-- Flags historical duplicate charges instead of deleting them (the app's own double_charges.sql\n-- report relies on seeing every duplicate), enforces one inactive payment per UPI ref going forward.\n-- and grandfathers existing orphan/null legacy rows (already surfaced by unattributed_payments.sql\n-- via NOT VALID constraints instead of deleting or fabricating data for them.\n\nALTER TABLE payments ADD

Edge cases covered

Caught in rehearsal (shown on ledgerly)	Refused by pgwarden, even with human approval
Passes CI/UAT, fails on real data (FK, NOT NULL on legacy rows)	Effects on the card differ from reality: EFFECTS_MISMATCH , rolled back
Silent cascade deletes (payments → refunds)	SQL changed after rehearsal, even one character: REHEARSAL_MISMATCH
Half-applied deploys (applies are one transaction instead)	No rehearsal, or a failed one: REHEARSAL_NOT_FOUND / _FAILED
Legacy data grandfathered, not deleted (NOT VALID , partial index)	DROP / TRUNCATE / RENAME: POLICY_REFUSED
The agent's own destructive fix (−119 payments) exposed on the card	Rows lost in protected tables: PROTECTED_ROWS_LOST
Prompt injection in PR text and comments, treated as untrusted	Prod changed since rehearsal: DRIFT_DETECTED ; approval after 30 min: REHEARSAL_STALE
PII leaving prod (5 columns masked); 0 credentials in the sandbox	Two applies at once: APPLY_IN_PROGRESS ; backup before every apply

Known limits

- The agent's fixes vary between runs. Before protected tables existed, two destructive fixes were approved in testing.
- Effects count rows and schema, not changed values.
- Full-table copies suit demo scale only.
- One runner per repo, so jobs queue.
- An applied fix isn't yet written back to the PR's migration file.